# C++ while 循环语句

## 基础知识专题讲义与配套练习

目标：掌握基础知识点与最基本应用

**严格按入门难度设计**

10 道单项选择题 + 5 道多项选择题 + 5 道填空题 + 4 道基础解答题

共 24 道题，答案与详细解析统一置于题目之后

核验来源：C++ 工作草案、cppreference、Microsoft Learn

生成日期：2026 年 8 月 20 日

# 目录


**1 使用说明 3**

**2 图片内容重新编码 3**

2.1 图片 1：while 的作用、语法与打印示例 . . . . . 3

2.2 图片 2–3：猜数字示例 . . . . . 4

**3 原图问题与必要纠错 5**

3.1 图片 1 打印示例的错误 . . . . . 5

3.2 猜数字示例的错误与逻辑缺口 . . . . . 5

**4 知识点清单 7**

**5 核心概念详解 7**

5.1 知识点 1：什么是 while 循环 . . . . . 7

5.2 知识点 2：while 的执行流程 . . . . . 8

5.3 知识点 3：计数循环的三个核心部件 . . . . . 9

5.4 知识点 4：为什么会出现死循环 . . . . . 9

5.5 知识点 5：while (true)、while (1) 与 break . . . . . 10

5.6 知识点 6：猜数字中的三分支判断 . . . . . 10

5.7 知识点 7：最基础输入输出 . . . . . 11

5.8 知识点 8：图片中的随机数三行代码 . . . . . 11

**6 最基础示例 11**

6.1 示例 1：打印 0、1、2 . . . . . 11

6.2 示例 2：条件一开始就是假 . . . . . 12

6.3 示例 3：用固定答案理解猜数字 . . . . . 12

**7 知识点—题目对应表 12**

**8 单项选择题（10 道） 14**

**9 多项选择题（5 道） 16**

**10 填空题（5 道） 17**

**11 基础解答题（4 道） 17**

**12 答案速查表 19**

12.1 单项选择题 . . . . . 19

12.2 多项选择题 . . . . . 19

12.3 填空题 . . . . . 19

**13 单项选择题详细解析 19**

**14 多项选择题详细解析 22**

**15 填空题详细解析 23**

**16 基础解答题详细解析 24**

**17 高频易错点总结 27**

**18 基础解题步骤总结 28**

18.1 看到计数型 `while`，按 4 步检查 . . . . . 28

18.2 看到猜数字程序，按 4 个关系检查 . . . . . 28

**19 参考资料与核验来源 28**

**20 质量检查记录 30**

# 1 使用说明

本资料只服务于一个目标：**把图片中出现的 while 循环知识真正学会，并能完成最基本的代码阅读、补全和简单编写**。题目不追求技巧，不涉及竞赛算法，也不要求复杂推理。

**推荐使用顺序：**

1. 先阅读“图片内容重新编码”和“原图问题与必要纠错”，知道笔记原来写了什么、哪里需要改；
2. 再阅读“核心知识详解”，重点掌握“先判断、再执行、再回到条件”的循环过程；
3. 独立完成 24 道题，不提前查看答案；
4. 最后逐题对照解析，尤其关注“为什么其他选项错”和“如何避免死循环”。

本资料**不嵌入原始照片**，而是将照片内容重新排版为可复制文本和代码。对于笔迹中不能百分之百确认的字符，均单独说明，不把猜测伪装成原文。

# 2 图片内容重新编码

## 2.1 图片 1：while 的作用、语法与打印示例

图片顶部手写标题可辨认为“第 2：循环结构”，随后为“while 语句”。主要内容如下：

1. **作用：**满足循环条件，执行循环语句。
2. **语法：**while（循环条件）{循环语句}。
3. **解释：**只要循环条件的结果为真，就执行循环语句。
4. **示例代码：**打印 0–9 数字。

图片中的手写代码按可辨识内容转录如下。此处**保留原图中的大小写、标点和可能缺失的分号**，后面再统一纠错。

```
#include <iostream>
Using namespace Std;

int main()
{
    int num = 0;
    while (num <= 9)
    {
        cout << "我是: " << num << endl;
        num++
    }
    return 0;
}
```

图片底部红笔备注可辨认为：

- “while() 中填入循环条件”；
- “注：写循环时，一定要避免死循环的出现”。

## 2.2 图片 2–3：猜数字示例

第二张图片顶部写有“C++”“第 2 循环语句”“一、while 语句”“2. 实例：猜数字”。代码从图片 2 延续到图片 3。

以下为按图片顺序重新编码的版本。为保持忠实，暂不把它修成可运行程序。

```
#include <iostream>.
Using namespace Std;

int main.()
{
    // 添加随机数种子
    srand((unsigned int) time(NULL));

    // 生成随机数 1~100
    int num = rand() % 100 + 1

    // 猜测数字
    while (1)
    {
        int guess;
        cout << "请输入你猜测的数字(1-100)：" << endl;
        cin >> guess

        if (guess < num)
        {
            cout << "您输入的数字过小，请重新输入！" << endl;
        }
        else if (guess > num)
        {
            cout << "您输入的数字过大，请重新输入！" << [原图此处疑似 endl];
        }
        else
        {
            cout << "恭喜您输入正确！" << [原图此处疑似 endl];
        }
        return 0;
    }
```

**无法完全可靠辨认处：**图片 3 两处换行操纵符的手写字形近似“enal/endl”。结合图片 1 已明确使用 endl、并结合 C++ 输出语句上下文，**可以合理推测作者意图为 endl**；但为遵守“不把推测当原文”的原则，上面的原始转录仍标为“疑似 endl”。规范代码中使用正确的 std::endl。

# 3 原图问题与必要纠错

## 3.1 图片 1 打印示例的错误

| 原图写法 | 问题 | 规范写法与理由 |
| --- | --- | --- |
| `Using namespace Std;` | C++ 关键字和命名空间名称区分大小写；`Using`、`Std` 均不正确。 | 若沿用该风格，应写成 `using namespace std;`。更规范的教学版也可直接在对象前写 `std::`。 |
| `num++` 后无分号 | 表达式语句需要以分号结束。 | 写成 `num++;`。 |
| `while (num <= 9)` | 这一行本身正确。 | 当 `num` 从 0 开始、每轮加 1 时，条件依次对 0–9 为真，对 10 为假，因此刚好执行 10 次。 |

修正后的可运行版本如下：

```cpp
#include <iostream>

int main()
{
    int num = 0;

    while (num <= 9)
    {
        std::cout << "我是: " << num << std::endl;
        num++;
    }

    return 0;
}
```

## 3.2 猜数字示例的错误与逻辑缺口

| 原图写法/结构 | 问题 | 修正方法 |
| --- | --- | --- |
| `#include <iostream>.` | 末尾的点不是预处理指令的一部分。 | 写成 `#include <iostream>`。 |
| `Using namespace Std;` | 大小写错误。 | 写成 `using namespace std;`，或使用 `std::cout`、`std::cin` 等。 |
| `int main.()` | `main` 与圆括号之间不应有点。 | 写成 `int main()`。 |
| 只包含 `<iostream>` 却调用 `rand/srand/time` | 这些函数分别属于 C 风格随机数与时间接口，规范 C++ 代码应包含相应头文件。 | 增加 `<cstdlib>` 与 `<ctime>`。[3,4,5] |

| 原图写法/结构 | 问题 | 修正方法 |
| --- | --- | --- |
| `int num = rand() % 100 + 1` 后缺分号 | 语句未结束，会导致编译错误。 | 末尾添加分号。 |
| `cin >> guess` 后缺分号 | 同上。 | 写成 `std::cin >> guess;`。 |
| `while (1)` | 语法上可以形成恒真循环，不是语法错误；但必须有明确退出机制。 | 初学者更推荐写 `while (true)`，并在猜对时执行 `break;`。[1,2] |
| 猜对分支没有 `break;` | 猜对后循环不会因为条件自动变假。 | 在“恭喜”输出后添加 `break;`。 |
| `return 0;` 的位置与大括号不完整 | 按图片现有括号关系，程序结构不完整；若把 `return 0;` 放在循环体里，又会导致第一次猜完就直接结束程序。 | 应先结束 `while` 循环，再在 `main` 末尾 `return 0;`。 |

规范、可运行且保持原笔记思路的版本如下：

```
#include <cstdlib>
#include <ctime>
#include <iostream>

int main()
{
    // 使用当前时间为伪随机数生成器设置一次种子
    std::srand(static_cast<unsigned int>(std::time(nullptr)));

    // 生成 1~100 的整数（适合本入门示例）
    int num = std::rand() % 100 + 1;

    while (true)
    {
        int guess;
        std::cout << "请输入你猜测的数字（1~100）：" << std::endl;
        std::cin >> guess;

        if (guess < num)
        {
            std::cout << "您输入的数字过小，请重新输入！" << std::endl;
        }
        else if (guess > num)
        {
            std::cout << "您输入的数字过大，请重新输入！" << std::endl;
        }
        else
        {
            std::cout << "恭喜您输入正确！" << std::endl;
```

```
            break;
        }
    }

    return 0;
}
```

**关于 `rand() % 100 + 1`：**它非常适合本次“理解 `while`”的入门练习。`rand()` 返回非负伪随机整数，`% 100` 取余后得到 0–99，再加 1 得到 1–100。[3,4] 不过在现代 C++ 的正式程序中，通常更推荐 `<random>`；这属于后续知识，本资料不作为考查重点。

## 4 知识点清单

| 序号 | 知识点 | 本资料要求掌握到什么程度 |
|---|---|---|
| 1 | `while` 的作用 | 知道它用于“条件为真时重复执行”。 |
| 2 | 基本语法 | 会认出并写出 `while (条件) { 循环体 }`。 |
| 3 | 执行顺序 | 知道每一轮都是先检查条件，再决定是否执行循环体。 |
| 4 | 循环控制变量 | 会用“初始化 + 条件 + 更新”控制简单计数循环。 |
| 5 | 边界条件 | 能判断 `<` 与 `<=` 对循环次数的影响。 |
| 6 | 死循环 | 知道控制变量不更新可能导致条件一直为真。 |
| 7 | 恒真循环 | 知道 `while (true)` / `while (1)` 会持续循环，必须设计退出方式。 |
| 8 | `break` | 会在“猜对数字”后退出当前循环。 |
| 9 | `cin`/`cout` | 会完成最基础的输入与输出。 |
| 10 | 猜数字逻辑 | 会判断“猜小、猜大、猜对”三个分支。 |
| 11 | `rand()`/`srand()`/`time()` | 只理解图片示例中它们的基本作用，不要求深入随机数理论。 |

## 5 核心概念详解

### 5.1 知识点 1：什么是 `while` 循环

**基本定义：**在 C++ 中，`while` 是一种循环语句。它会在每次执行循环体之前检查条件；当条件为真时执行循环体，执行完后回到条件处再次检查；当条件为假时结束循环。C++ 工作草案明确规定，`while` 的条件检查发生在每次循环体执行之前。[1,2]

**通俗解释：**把它理解成一句话：

> **“只要条件还成立，就再做一遍。”**

**基本语法：**

```
while（循环条件）
{
    循环语句;
}
```

这里有两个最重要部分：

- **循环条件：**写在圆括号 `()` 中，决定“还要不要继续”；
- **循环体：**通常写在大括号 `{}` 中，是“每一轮真正执行的内容”。

**适用条件：**当你知道“什么时候停止”，但不一定想用固定的 `for` 写法时，`while` 很自然。对于本资料，只需掌握计数循环和猜数字循环。

**极简示例：**

```
int n = 1;
while (n <= 3)
{
    std::cout << n << std::endl;
    n++;
}
```

输出为 1、2、3。因为 `n=1,2,3` 时条件都成立；当 `n` 增加到 4 后，`4 <= 3` 为假，循环停止。

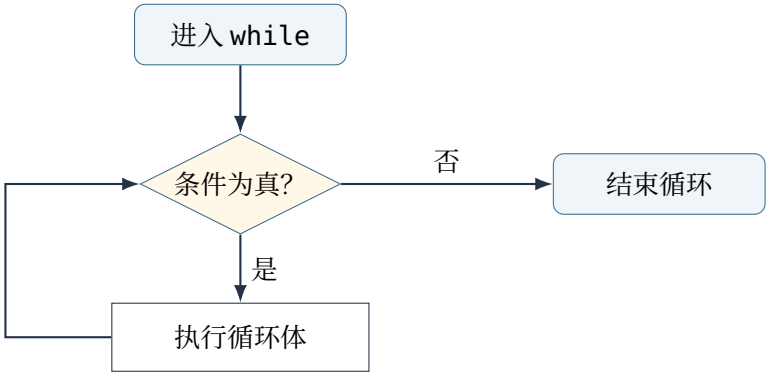
**常见错误：**把 `while` 误认为“先执行一次再判断”。这是 `do...while` 的特点，不是 `while` 的特点。

**记忆提示：while = 先问“行不行”，再决定“做不做”。**

## 5.2 知识点 2：`while` 的执行流程

标准语义可以简化为以下 4 步：[1,2]

1. 检查循环条件；
2. 条件为假：立刻离开循环；
3. 条件为真：执行循环体；
4. 循环体结束后回到第 1 步。




**最重要的结论：**如果条件一开始就是假，循环体可以一次都不执行。

### 5.3 知识点 3：计数循环的三个核心部件

以打印 0–9 为例：

```
int num = 0;           // (1) 初始化
while (num <= 9)       // (2) 条件
{
    std::cout << num << std::endl;
    num++;             // (3) 更新
}
```

1. **初始化：** `int num = 0;` 决定从哪里开始；
2. **条件：** `num <= 9` 决定什么时候继续；
3. **更新：** `num++` 让 `num` 逐渐接近停止条件。

**最基础的使用步骤：**

1. 先确定起始值；
2. 再确定最后允许进入循环的值；
3. 最后保证每轮都让控制变量发生正确变化。

**边界检查：**

- `num < 9`：最后执行到 8；
- `num <= 9`：最后执行到 9。

**记忆提示：从哪开始？到哪还算真？每轮怎么变？**

### 5.4 知识点 4：为什么会出现死循环

“死循环”在入门语境下，通常指循环条件持续为真，程序无法按预期离开循环。

典型错误：

```
int num = 0;
while (num <= 9)
{
    std::cout << num << std::endl;
    // 忘了 num++;
}
```

由于 `num` 始终是 0，条件 `num <= 9` 始终为真，于是循环一直重复。

**检查死循环的最简单问题：**“循环体执行一轮之后，条件有没有可能变成假？”如果答案是“没有”，就要检查控制变量是否忘记更新，或是否缺少计划中的 `break`。

### 5.5 知识点 5：while (true)、while (1) 与 break

在 C++ 条件位置，表达式会被按布尔意义判断。true 就是真；非零整数 1 也可转换为真，因此 while (1) 和 while (true) 都可构成恒真循环。[1,2]

对初学者，推荐：

```
while (true)
{
    // 重复执行
    if (满足退出条件)
    {
        break;
    }
}
```

break; 的作用是**立即退出当前循环**。在猜数字程序里，“猜对”就是退出条件。

**记忆提示：恒真循环负责“持续问”，break 负责“满足条件就停”。**

### 5.6 知识点 6：猜数字中的三分支判断

设正确答案为 num，用户输入为 guess：

- guess < num：猜小了；
- guess > num：猜大了；
- 既不小也不大，即 guess == num：猜对了。

对应代码：

```
if (guess < num)
{
    // 提示“过小”
}
else if (guess > num)
{
    // 提示“过大”
}
else
{
    // 相等：猜对
    break;
}
```

**常见错误：**把“小于”和“提示过大”写反。最直接的检查方法是代入一个具体数：如果正确答案 50、你输入 20，那么 20 小于 50，所以应该提示“过小”。

### 5.7 知识点 7：最基础输入输出

`<iostream>` 提供常用标准流对象；`std::cin` 用于从标准输入读取数据，`std::cout` 用于向标准输出写数据。cppreference 与 Microsoft Learn 均对此给出相同的基本定义。[6,7,8]

```
int guess;
std::cin >> guess;              // 输入
std::cout << guess << std::endl; // 输出
```

**符号解释：**

- `>>`：在这里表示从输入流提取数据到变量；
- `<<`：在这里表示把数据插入输出流；
- `std::endl`：结束当前行并刷新输出缓冲区。[9]

### 5.8 知识点 8：图片中的随机数三行代码

图片中涉及：

```
std::srand(static_cast<unsigned int>(std::time(nullptr)));
int num = std::rand() % 100 + 1;
```

只需掌握以下最低限度概念：

1. `std::srand(seed)`：给 `std::rand()` 使用的伪随机数生成器设置种子；同一种子会得到相同的伪随机序列。[3,4]
2. `std::time(nullptr)`：取得当前日历时间的一个时间值，可用于产生变化的种子。[5]
3. `std::rand()`：得到一个从 0 到 `RAND_MAX` 的非负伪随机整数。[4]
4. `% 100`：取除以 100 的余数，所以结果落在 0–99；
5. 再 `+1`：把范围平移到 1–100。

**“公式”式理解：**

$$\texttt{num} = \big(\texttt{rand()} \bmod 100\big) + 1.$$

其中 $\bmod 100$ 表示“取除以 100 的余数”。这里不是数学竞赛中的复杂模运算，只需理解余数范围是 0–99。

## 6 最基础示例

### 6.1 示例 1：打印 0、1、2

```
int n = 0;
while (n <= 2)
{
    std::cout << n << std::endl;
    n++;
```

```
}
```

**逐轮观察：**

| 进入本轮前的 n | 条件 n <= 2 | 是否输出 | 本轮结束后的 n |
|---|---|---|---|
| 0 | 真 | 输出 0 | 1 |
| 1 | 真 | 输出 1 | 2 |
| 2 | 真 | 输出 2 | 3 |
| 3 | 假 | 不执行循环体 | – |

### 6.2 示例 2：条件一开始就是假

```
int n = 5;
while (n < 3)
{
    std::cout << n << std::endl;
}
```

因为第一次检查时 5 < 3 就是假，所以循环体执行 0 次。

### 6.3 示例 3：用固定答案理解猜数字

先不使用随机数，把正确答案固定为 5：

```
int num = 5;
while (true)
{
    int guess;
    std::cin >> guess;

    if (guess < num)
        std::cout << "过小" << std::endl;
    else if (guess > num)
        std::cout << "过大" << std::endl;
    else
    {
        std::cout << "正确" << std::endl;
        break;
    }
}
```

若依次输入 3、7、5，则依次得到“过小”“过大”“正确”，随后退出循环。

## 7 知识点—题目对应表

| 题号 | 题型 | 对应知识点 | 难度 | 主要能力 |
|---|---|---|---|---|
| 1 | 单选 | `while` 基本语法 | 入门 | 语法辨认 |
| 2 | 单选 | 条件为真时的行为 | 入门 | 概念识别 |
| 3 | 单选 | 先判断后执行 | 入门 | 执行次数判断 |
| 4 | 单选 | 计数循环追踪 | 入门 | 一步模拟 |
| 5 | 单选 | 死循环与变量更新 | 入门 | 错误识别 |
| 6 | 单选 | 恒真循环 | 入门 | 概念识别 |
| 7 | 单选 | `break` | 入门 | 退出机制 |
| 8 | 单选 | `rand()%100+1` | 基础 | 范围判断 |
| 9 | 单选 | `cin` 输入 | 入门 | 语句辨认 |
| 10 | 单选 | `<iostream>` | 入门 | 头文件识别 |
| 11 | 多选 | `while` 语义 | 入门 | 逐项判断 |
| 12 | 多选 | 避免死循环 | 入门 | 错误预防 |
| 13 | 多选 | 猜数字逻辑 | 入门 | 分支关系 |
| 14 | 多选 | 原图代码纠错 | 基础 | 语法检查 |
| 15 | 多选 | 输入输出 | 入门 | 基础语句判断 |
| 16 | 填空 | `while` 条件位置 | 入门 | 语法记忆 |
| 17 | 填空 | 循环结束后的变量值 | 基础 | 简单追踪 |
| 18 | 填空 | 首次条件为假 | 入门 | 执行次数 |
| 19 | 填空 | 随机整数范围 | 基础 | 简单范围 |
| 20 | 填空 | `break` | 入门 | 关键字记忆 |
| 21 | 解答 | 循环追踪 | 基础 | 步骤分析 |
| 22 | 解答 | 编写计数循环 | 基础 | 模仿编程 |
| 23 | 解答 | 修复死循环 | 基础 | 错误修正 |
| 24 | 解答 | 固定答案猜数字 | 基础 | 轻度综合 |

## 8 单项选择题（10 道）

**说明：**每题只有一个正确答案。

**1.** 在 C++ 中，`while` 的循环条件应写在什么位置？

A. 大括号 `{}` 后面

B. `while` 后面的圆括号 `()` 中

C. `return` 后面

D. `#include` 后面

**2.** 当 `while` 的循环条件为真时，程序会怎样？

A. 立即结束整个程序

B. 跳过循环体

C. 执行循环体，然后再次检查条件

D. 自动把条件改成假

**3.** 若下面代码中的条件第一次检查就是假，循环体会执行多少次？

```
while（条件）
{
    循环体;
}
```

A. 0 次

B. 1 次

C. 2 次

D. 一定无限次

**4.** 下面代码的输出顺序是：

```
int num = 0;
while (num <= 2)
{
    std::cout << num << " ";
    num++;
}
```

A. 1 2 3

B. 0 1

C. 0 1 2 3

D. 0 1 2

**5.** 下面代码最可能出现什么问题？

```
int num = 0;
while (num <= 9)
{
    std::cout << num << std::endl;
}
```

A. num 会自动加 1

B. 可能形成死循环，因为 num 没有更新

C. 循环一次都不会执行

D. while 只能循环 9 次

**6.** while (1) 在图片的猜数字程序中主要表示什么?

A. 条件恒假

B. 只执行一次

C. 条件恒真，需要另设退出方式

D. 表示随机数 1

**7.** 在规范的猜数字程序中，猜对后可用哪个语句退出当前 while 循环?

A. break;

B. include;

C. int;

D. cout;

**8.** 对图片中的表达式 rand() % 100 + 1，按本入门示例理解，其整数范围是:

A. 0–100

B. 0–99

C. 1–99

D. 1–100

**9.** 若要把键盘输入的整数保存到变量 guess，哪一行最合适?

A. std::cout >> guess;

B. std::cin >> guess;

C. std::cin << guess;

D. std::endl >> guess;

**10.** 图片中的 cin、cout 主要由哪个头文件提供?

A. <ctime>

B. <cstdlib>

C. `<iostream>`

D. `<random>`

## 9 多项选择题（5 道）

**说明：**每题有 2 个或 3 个正确选项。**全部选对得分；选错不得分**。

**11.** 关于 C++ 的 `while` 循环，下列说法正确的是：

A. 循环条件写在 `while` 后的圆括号中

B. 条件为真时会执行循环体

C. 条件为假时会退出循环

D. 循环体一定至少执行一次

**12.** 下列哪些做法有助于避免“本来应该结束却一直循环”的问题？

A. 让控制变量在循环中正确更新

B. 设计一个最终能够变成假的循环条件

C. 使用恒真循环时，在合适位置设置 `break`

D. 只要写成 `while (true)` 就一定不会死循环

**13.** 设正确答案为 `num`，用户输入为 `guess`。下列判断正确的是：

A. `guess < num` 时应提示“过小”

B. `guess > num` 时应提示“过大”

C. `guess == num` 时可以提示正确并退出循环

D. `rand() % 100 + 1` 的结果范围是 0–99

**14.** 根据上传图片中的原始代码，下列哪些属于需要修正的问题？

A. `Using namespace Std;` 的大小写不规范

B. 使用 `rand/srand/time` 时缺少相应头文件

C. 猜对后缺少明确的循环退出逻辑

D. `while (1)` 在 C++ 中必然是语法错误

**15.** 关于最基础的 C++ 输入输出，下列说法正确的是：

A. `std::cout` 可用于标准输出

B. `std::cin` 可用于标准输入

C. `std::endl` 可结束一行并刷新输出缓冲区

D. `std::cin << guess;` 是把键盘输入读入 `guess` 的标准写法

## 10 填空题（5 道）

**16.** `while (_____)` 的圆括号中应填写循环的_____。

**17.** 执行下面代码后，循环结束时 `num` 的值为_____。

```
int num = 0;
while (num <= 9)
{
    num++;
}
```

**18.** 若一个 `while` 条件在第一次检查时就是假，则循环体执行_____ 次。

**19.** 在本资料的基础写法中，`rand() % 100 + 1` 的取值范围为：最小值_____，最大值_____。

**20.** 在恒真循环中，若满足退出条件，可使用关键字_____ 立即退出当前循环。

## 11 基础解答题（4 道）

### 第 21 题：追踪一个最简单的 `while` 循环

阅读代码：

```
int n = 1;
while (n <= 3)
{
    std::cout << n << " ";
    n++;
}
```

回答：

1. 程序依次输出哪些数字？
2. 循环结束后 `n` 的值是多少？

### 第 22 题：编写打印 0–4 的循环

请使用 `while` 循环编写一段代码，按顺序输出 0、1、2、3、4。要求使用变量 `num`，并让它每轮增加 1。

### 第 23 题：修复一个死循环

下面的代码会不断输出 0：

```
int n = 0;
while (n < 3)
{
    std::cout << n << " ";
```

```
}
```

请指出原因，并只添加一**条最关键的语句**，使程序输出 `0 1 2` 后结束。

### 第 24 题：固定答案版“猜数字”

不使用随机数，设正确答案固定为 5。请补全/写出一个最基础的猜数字循环：

- 输入小于 5：输出“过小”；
- 输入大于 5：输出“过大”；
- 输入等于 5：输出“正确”，并退出循环。

要求使用 `while (true)`、`if / else if / else` 和 `break`。

# 12 答案速查表

## 12.1 单项选择题

| 题号 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| 答案 | B | C | A | D | B | C | A | D | B | C |

## 12.2 多项选择题

| 题号 | 11 | 12 | 13 | 14 | 15 |
|---|---|---|---|---|---|
| 答案 | ABC | ABC | ABC | ABC | ABC |

## 12.3 填空题

| 题号 | 标准答案 |
|---|---|
| 16 | 循环条件（或“条件”）；第二空同样为“条件/循环条件” |
| 17 | 10 |
| 18 | 0 |
| 19 | 1；100 |
| 20 | `break`（写 `break;` 也接受） |

# 13 单项选择题详细解析

## 第 1 题

**正确答案：B　考点：** `while` 基本语法　**难度：** 入门

**相关定义：**基本形式为 `while (condition) statement`；条件位于 `while` 后的圆括号中。[1,2]

**判断过程：**

- A 错：大括号通常包围循环体，不是写条件的位置。
- B 对：圆括号内就是每轮执行前要检查的条件。
- C 错：`return` 用于从函数返回，不是 `while` 条件位置。
- D 错：`#include` 是预处理指令，与循环条件无关。

**常见误区：**把“条件”和“循环体”位置混淆。

**一句总结：`while` 后看圆括号，圆括号里就是条件。**

## 第 2 题

**正确答案：C　考点：**条件为真时的执行流程　**难度：**入门

**相关规则：**每次先检查条件；条件为真就执行循环体，之后再回到条件。[1,2]

- A 错：条件为真不是“结束程序”，而是“继续循环”。
- B 错：条件为真不会跳过循环体。
- C 对：这是 `while` 最核心的执行过程。
- D 错：C++ 不会自动替你修改条件中相关变量；是否变假取决于程序逻辑。

**常见误区：**误以为 `while` 会自动计数。实际上像 `num++` 这样的更新语句必须自己写。

## 第 3 题

**正确答案：A　考点：**`while` 是前测试循环　**难度：**入门

**解题思路：**第一次进入循环体之前就要检查条件。若第一次就是假，则没有机会执行循环体。

- A 对：执行 0 次。
- B 错：至少执行 1 次的是 `do...while` 的典型特征，不是本题的 `while`。
- C 错：没有任何依据会执行 2 次。
- D 错：条件为假恰好会停止，而不是无限循环。

**一句总结：`while` 可以一次都不执行。**

## 第 4 题

**正确答案：D　考点：**计数循环追踪　**难度：**入门

**逐步判断：**

1. `num=0`，`0<=2` 为真，输出 0，然后变 1；
2. `num=1`，条件为真，输出 1，然后变 2；
3. `num=2`，条件为真，输出 2，然后变 3；
4. `num=3`，`3<=2` 为假，停止。

所以输出 `0 1 2`。

**选项判断：**A 忽略了起始值 0；B 漏掉 2；C 错把停止后的 3 也输出；D 正确。

**常见误区：**看到循环结束时变量变成 3，就误以为 3 也会被输出。实际上先检查 `3<=2`，已经是假。

### 第 5 题

**正确答案：B　考点：**死循环与变量更新　**难度：**入门

**关键观察：**`num` 初始化为 0，但循环体中没有任何语句改变 `num`。所以 `num <= 9` 一直保持为真。

- A 错：变量不会因为处于 `while` 中自动增加。
- B 对：条件一直为真，程序会持续输出 0。
- C 错：第一次 `0<=9` 就是真，所以会执行。
- D 错：`while` 没有“最多 9 次”的固定限制。

**一句总结：计数循环别忘了更新控制变量。**

### 第 6 题

**正确答案：C　考点：**恒真循环　**难度：**入门

整数 1 在条件判断中表示真，因此 `while (1)` 会一直满足条件，除非循环内部通过 `break`、`return` 等机制离开。对初学者可写成更直观的 `while (true)`。[1,2]

A 错在把 1 当作假；B 错在“恒真”不等于只执行一次；C 正确；D 把循环条件 1 和随机数概念混淆。

### 第 7 题

**正确答案：A　考点：**`break`　**难度：**入门

`break;` 用于离开当前循环。猜数字程序在“猜对”时使用它，正好表达“目标已完成，不必继续问”。

B、C、D 都不是退出循环的语句：`include` 是预处理相关词；`int` 是类型关键字；`cout` 是输出对象。

### 第 8 题

**正确答案：D　考点：**简单随机数范围　**难度：**入门

**使用的规则：**`rand()` 产生非负整数；对 100 取余的余数只能是 0–99；最后加 1。[4]

$$0 \le \texttt{rand()}\%100 \le 99 \quad \Longrightarrow \quad 1 \le \texttt{rand()}\%100 + 1 \le 100.$$

因此范围是 1–100。

A 多包含了 0；B 忘了最后的 `+1`；C 错把 100 排除；D 正确。

### 第 9 题

**正确答案：B　考点：** `std::cin` 输入　**难度：** 入门

`std::cin` 控制标准输入，常用提取运算符 `>>` 把读到的数据保存到变量中。[6,8]

所以 `std::cin >> guess;` 正确。A 把输出对象和输入方向混在一起；C 运算符方向错误；D 中 `std::endl` 不是输入流。

### 第 10 题

**正确答案：C　考点：** `<iostream>`　**难度：** 入门

`<iostream>` 声明标准输入输出流对象，包括 `cin` 和 `cout`。[6,7,8]

A 的 `<ctime>` 与时间相关；B 的 `<cstdlib>` 包含 `rand/srand` 等 C 风格通用函数；D 的 `<random>` 是现代随机数库。只有 C 与本题直接对应。

## 14　多项选择题详细解析

### 第 11 题

**正确答案：ABC　考点：** `while` 基本语义　**难度：** 入门

- **A：正确**。条件写在 `while(...)` 的圆括号内。
- **B：正确**。条件为真时执行循环体。
- **C：正确**。条件为假时循环停止。
- **D：错误**。`while` 在进入循环体之前先判断；若第一次就是假，可以执行 0 次。[1,2]

**总结：**ABC 正确。

### 第 12 题

**正确答案：ABC　考点：**避免死循环　**难度：** 入门

- **A：正确**。例如 `num++` 能让控制变量向停止边界变化。
- **B：正确**。正常计数循环必须让条件最终有机会变为假。
- **C：正确**。若设计成 `while(true)`，就应在目标完成时用 `break` 等退出。
- **D：错误**。`while(true)` 本身正是恒真条件，如果没有退出逻辑反而会一直循环。

**总结：**不要只看“写了什么条件”，还要看“一轮之后条件能否改变”。

### 第 13 题

**正确答案：ABC　考点：**猜数字分支逻辑　**难度：**入门

- **A：正确。**猜测值小于答案，说明猜小了。
- **B：正确。**猜测值大于答案，说明猜大了。
- **C：正确。**相等时目标完成，可输出成功信息并 `break`。
- **D：错误。**`rand()%100` 才是 0–99；再 `+1` 后是 1–100。

### 第 14 题

**正确答案：ABC　考点：**原图代码纠错　**难度：**基础

- **A：正确。**C++ 区分大小写，应为 `using namespace std;`，或直接使用 `std::`。
- **B：正确。**规范代码中 `rand/srand` 对应 `<cstdlib>`，`time` 对应 `<ctime>`。[3,4,5]
- **C：正确。**恒真循环要在“猜对”时退出，否则会继续循环。
- **D：错误。**`while(1)` 可以形成恒真循环，不是语法错误。

### 第 15 题

**正确答案：ABC　考点：**基础输入输出　**难度：**入门

- **A：正确。**`std::cout` 控制标准输出。[7]
- **B：正确。**`std::cin` 控制标准输入。[6]
- **C：正确。**`std::endl` 写入换行并刷新流。[9]
- **D：错误。**读取到变量应使用 `std::cin >> guess;`，不是 `<<`。

## 15　填空题详细解析

### 第 16 题

**标准答案：**第一空“循环条件/条件”；第二空“循环条件/条件”。

**定义：**`while` 后的圆括号保存控制本轮是否执行的条件。[1,2]

**格式提醒：**本题考的是位置与名称，不要求写具体表达式。写“条件”“循环条件”均接受。

### 第 17 题

**标准答案：**10

**步骤：**

1. 初始 `num=0`；
2. 当 `num=0,1,…,9` 时，`num<=9` 都是真，每轮执行 `num++`；
3. 当第 10 次更新完成后，`num` 变为 10；
4. 再检查 `10<=9` 为假，循环停止。

**易错点：**“最后一次进入循环时的值”是 9，但“循环结束后的值”是 10。

### 第 18 题

**标准答案：**0

**规则：**`while` 的条件检查在循环体执行之前。第一次就是假，则循环体没有任何一次执行机会。[1,2]

### 第 19 题

**标准答案：**1；100

**使用的表达式：**`rand() % 100 + 1`。

**符号说明：**`%` 是整数取余。对 100 取余的余数为 0–99；加 1 后为 1–100。

**可接受答案：**“最小 1，最大 100”“1 和 100”均可。

### 第 20 题

**标准答案：**`break`（写 `break;` 也正确）

**用途：**`break` 立即结束当前循环。在猜数字里，猜对后已经没有继续询问的必要，所以使用它最直接。

## 16 基础解答题详细解析

### 第 21 题解析：追踪循环

**最终答案：** 输出 `1 2 3`；循环结束后 `n=4`。

**1. 题意分析：**已知 `n` 从 1 开始，条件是 `n<=3`，每轮先输出再 `n++`。要求找输出序列和最终变量值。

**2. 知识定位：**使用“`while` 先判断条件，再执行循环体”的规则，以及 `n++` 表示让 `n` 增加 1。

**3. 规则说明：**

- `n <= 3`：只有当 `n` 不超过 3 时才能进入本轮；

- n++：本轮结束前把 n 增加 1。

**4. 解题步骤：**

| n | n<=3 | 输出 | 更新后 |
|---|---|---|---|
| 1 | 真 | 1 | 2 |
| 2 | 真 | 2 | 3 |
| 3 | 真 | 3 | 4 |
| 4 | 假 | 不输出 | – |

**5. 结果整理：**输出为 1 2 3，结束时 n=4。

**6. 结果检查：**最后一次真正进入循环时 n=3；执行 n++ 后才变成 4，因此最终值 4 合理。

**7. 易错提醒：**不要把“最终值 4”误写进输出，n=4 时条件已为假。

**8. 方法总结：做循环追踪题就画四列：当前值—条件—本轮做什么—更新后值。**

## 第 22 题解析：打印 0–4

**参考答案：**

```
int num = 0;
while (num <= 4)
{
    std::cout << num << std::endl;
    num++;
}
```

**1. 题意分析：**要求从 0 开始，按顺序打印到 4，并且使用 while 和每轮加 1。

**2. 知识定位：**最基本的计数循环由“初始化、条件、更新”三部分组成。

**3. 各部分含义：**

- int num=0;：从 0 开始；
- num<=4：0、1、2、3、4 都允许进入；
- num++：每轮向下一个整数移动。

**4. 结果检查：**当 num=4 时仍可执行并输出 4；随后变 5，5<=4 为假，停止。刚好不多也不少。

**5. 易错提醒：**若写成 num<4，只能输出到 3；若忘记 num++，会一直输出 0。

**6. 方法总结：要包含终点 4，就用 <=4。**

## 第 23 题解析：修复死循环

**原因：**n 一直保持为 0，条件 n<3 始终为真。
**应添加：**n++;

修正代码：

```
int n = 0;
while (n < 3)
{
    std::cout << n << " ";
    n++;
}
```

**1. 题意分析：**原程序会不断输出 0，需要用最小修改让它打印 0、1、2 后结束。

**2. 知识定位：**循环控制变量必须在循环过程中发生变化，才可能让条件从真变成假。

**3. 解题步骤：**

1. 初始 `n=0`；
2. 要想下一轮变成 1，就必须更新 `n`；
3. 每轮加 1 最直接，因此加入 `n++;`；
4. 当 `n` 最终变 3 时，`3<3` 为假，循环结束。

**4. 结果检查：**输出正好是 `0 1 2`，没有 3，因为 `n=3` 时已不能进入循环。

**5. 易错提醒：**不要把 `n++` 放在循环外面；那样循环内部仍无法改变 `n`。

**6. 方法总结：看到计数型死循环，第一检查项就是“控制变量有没有更新”。**

## 第 24 题解析：固定答案版猜数字

**参考答案：**

```
#include <iostream>

int main()
{
    int num = 5;

    while (true)
    {
        int guess;
        std::cin >> guess;

        if (guess < num)
        {
            std::cout << "过小" << std::endl;
        }
        else if (guess > num)
        {
            std::cout << "过大" << std::endl;
        }
        else
        {
```

```
            std::cout << "正确" << std::endl;
            break;
        }
    }

    return 0;
}
```

**1. 题意分析：**答案固定为 5，需要反复读取用户输入，直到输入正好等于 5。

**2. 知识定位：**本题只组合三个已学基础点：`while(true)` 重复、`if/else if/else` 比较大小、`break` 猜对退出。

**3. 规则说明：**

- `guess < num`：用户数字更小；
- `guess > num`：用户数字更大；
- 否则只能是相等；
- 相等时执行 `break;`，结束当前 `while`。

**4. 示例代入：**若依次输入 3、7、5：

1. 3 小于 5，输出“过小”，继续循环；
2. 7 大于 5，输出“过大”，继续循环；
3. 5 等于 5，进入 `else`，输出“正确”，执行 `break`；
4. 循环结束，执行 `return 0;`。

**5. 结果检查：**只有猜对时才退出，符合“反复猜直到正确”的任务要求。

**6. 易错提醒：**

- 不要把“过小”和“过大”写反；
- 不要漏掉 `break;`；
- 不要把 `return 0;` 放在每次循环都会执行的位置，否则第一次猜完程序就结束。

**7. 方法总结：恒真循环 + 三分支比较 + 猜对 `break`，就是最基础的猜数字框架。**

## 17 高频易错点总结

1. **把 `Using`、`Std` 写成大写：**C++ 区分大小写，正确形式为 `using namespace std;`。
2. **漏分号：**如 `num++`、`cin >> guess`、变量定义后都要检查分号。
3. **忘记更新控制变量：**例如漏掉 `num++;`，很容易导致条件一直为真。
4. **把 `<` 和 `<=` 混淆：**`num<9` 不包含 9；`num<=9` 包含 9。
5. **误以为 `while` 至少执行一次：**如果第一次条件就是假，可以执行 0 次。
6. **恒真循环没有退出条件：**`while(true)` 需要结合 `break` 等明确退出逻辑。

7. **猜大、猜小提示写反：**用具体数字代入检查，比如答案 50、输入 20，显然是“过小”。
8. **猜对后忘记 `break`：**会继续要求输入，失去“猜对即结束”的逻辑。
9. **把 `return 0;` 放在循环内部：**如果每轮无条件执行，就会在第一次猜测后直接结束整个 `main`。
10. **调用函数却缺头文件：**图片随机数示例需要补上 `<cstdlib>`、`<ctime>`。

# 18 基础解题步骤总结

## 18.1 看到计数型 `while`，按 4 步检查

1. **初值：**变量从几开始?
2. **条件：**什么值还能进入循环?
3. **循环体：**本轮输出/计算什么?
4. **更新：**本轮结束后变量怎样变化？最终能否让条件变假?

## 18.2 看到猜数字程序，按 4 个关系检查

1. 输入值是否成功读入 `guess`；
2. `guess<num` 是否提示“过小”；
3. `guess>num` 是否提示“过大”；
4. 相等时是否输出成功并退出循环。

**本专题最短记忆版：**

- `while`：先判断，真就做，做完再判断；
- 计数循环：初值 + 条件 + 更新；
- 防死循环：问自己“条件最终怎么变假?”；
- 猜数字：小了提示小，大了提示大，等于就 `break`。

# 19 参考资料与核验来源

本资料的关键语义使用至少两个独立来源交叉核验；题目均围绕上传图片重新编制，没有直接复制网络题库。

[1] **C++ Working Draft（在线工作草案）**. “The while statement”, [stmt.while].
https://eel.is/c++draft/stmt.while
访问日期：2026-08-20。
**主要核验：**`while` 循环体重复执行直到条件为假；条件在每次循环体执行之前检查。

[2] **cppreference**. “while loop”.
https://en.cppreference.com/cpp/language/while
访问日期：2026-08-20。
**主要核验：**`while` 语法、条件转换为布尔意义、`break`/`continue` 与循环关系。

[3] **cppreference**. “std::srand”.
https://en.cppreference.com/cpp/numeric/random/srand
访问日期：2026-08-20。
**主要核验：**`srand` 为 `rand` 使用的伪随机数生成器设置种子、同一种子产生同一序列的基本性质。

[4] **Microsoft Learn**. “rand” / “srand”, Microsoft C/C++ Runtime Library Reference.
https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/rand?view=msvc-170
https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/srand?view=msvc-170
访问日期：2026-08-20。
**主要核验：**`rand` 返回伪随机整数、`srand` 设置初始种子；并提示此接口不适用于密码学随机需求。

[5] **cppreference**. “std::time”.
https://en.cppreference.com/cpp/chrono/c/time
访问日期：2026-08-20。
**主要核验：**`std::time(nullptr)` 返回当前日历时间编码，可作为基础示例中的变化种子来源。

[6] **cppreference**. “std::cin, std::wcin”.
https://en.cppreference.com/cpp/io/cin
访问日期：2026-08-20。
**主要核验：**`std::cin` 与标准输入关联。

[7] **cppreference**. “std::cout, std::wcout”.
https://en.cppreference.com/cpp/io/cout
访问日期：2026-08-20。
**主要核验：**`std::cout` 与标准输出关联。

[8] **Microsoft Learn**. “<iostream>”.
https://learn.microsoft.com/en-us/cpp/standard-library/iostream?view=msvc-170
访问日期：2026-08-20。
**主要核验：**`<iostream>` 声明标准流对象，包含常见的 `cin`、`cout` 等。

[9] **Microsoft Learn**. “<ostream> functions —endl”.
https://learn.microsoft.com/en-us/cpp/standard-library/ostream-functions?view=msvc-170
访问日期：2026-08-20。
**主要核验：**`endl` 写入换行并刷新缓冲区。

## 20 质量检查记录

| 检查项目 | 结果 | 说明 |
|---|---|---|
| 图片 3 张是否全部纳入内容识别 | 通过 | 图片 2–3 的猜数字代码按连续内容整理 |
| 原图无法完全辨认处是否标注 | 通过 | 两处疑似 `endl` 已明确标注 |
| 原图错误是否先说明再修正 | 通过 | 独立设置“原图问题与必要纠错” |
| 单项选择题是否恰好 10 道 | 通过 | 第 1–10 题 |
| 多项选择题是否恰好 5 道 | 通过 | 第 11–15 题，每题 2–3 个正确项 |
| 填空题是否恰好 5 道 | 通过 | 第 16–20 题 |
| 基础解答题是否恰好 4 道 | 通过 | 第 21–24 题 |
| 总题量是否恰好 24 道 | 通过 | 10+5+5+4=24 |
| 题目难度是否保持入门/基础 | 通过 | 无竞赛题、复杂算法题、长计算题 |
| 答案是否集中放在全部题目之后 | 通过 | 题目结束后才出现答案速查与解析 |
| 单选错误选项是否逐项解释 | 通过 | 10 题均逐项说明 |
| 多选是否逐项分析 A–D | 通过 | 5 题均逐项说明 |
| 解答题是否给出步骤、检查、易错点 | 通过 | 4 题均给出结构化解析 |
| 关键知识是否多源核验 | 通过 | 工作草案 + cppreference + Microsoft Learn |

**编译与代码验证说明：**本文件已使用 XeLaTeX 实际编译；两份规范版 C++ 示例已使用 C++17 编译器进行语法检查并通过。最终 PDF 已在生成流程中完成页面渲染、可打开性与版面越界检查。